

NIM : 32230107

Nama : Steven Setiawan

Diketahui bitmap 24 bit dalam nilai integer sbb:

\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4
\$001267F4	\$001267F4	\$004F20FF	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$00FF005A	\$001267F4
\$001267F4	\$001267F4	\$004F20FF	\$004F20FF	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$00FF005A	\$00FF005A	\$001267F4
\$001267F4	\$001267F4	\$004F20FF	\$004F20FF	\$004F20FF	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$00FF005A	\$00FF005A	\$001267F4
\$001267F4	\$001267F4	\$004F20FF	\$004F20FF	\$004F20FF	\$004F20FF	\$001267F4	\$001267F4	\$00FF005A	\$00FF005A	\$00FF005A	\$00FF005A	\$001267F4
\$001267F4	\$001267F4	\$004F20FF	\$004F20FF	\$004F20FF	\$004F20FF	\$004F20FF	\$004F20FF	\$00FF005A	\$00FF005A	\$00FF005A	\$00FF005A	\$001267F4
\$001267F4	\$001267F4	\$004F20FF	\$004F20FF	\$004F20FF	\$004F20FF	\$001267F4	\$001267F4	\$00FF005A	\$00FF005A	\$00FF005A	\$00FF005A	\$001267F4
\$001267F4	\$001267F4	\$004F20FF	\$004F20FF	\$004F20FF	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$00FF005A	\$00FF005A	\$00FF005A	\$001267F4
\$001267F4	\$001267F4	\$004F20FF	\$004F20FF	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$00FF005A	\$00FF005A	\$001267F4
\$001267F4	\$001267F4	\$004F20FF	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$00FF005A	\$00FF005A	\$001267F4
\$001267F4	\$001267F4	\$004F20FF	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$00FF005A	\$00FF005A	\$001267F4
\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$00FF005A	\$001267F4
\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4	\$001267F4

Lakukan kuantisasi terhadap channel R, G dan B jika citra tersebut dikonversi menjadi

1. Citra RGB 3 bit dengan nilai threshold 0 s/d 127 menjadi 0 dan 128 s/d 255 menjadi 1

	R	G	B									
\$001267F4	18	103	244	0	0	1						
\$004F20FF	79	32	255	0	0	1						
\$00FF005A	255	0	90	1	0	0						
Citra 3-bit												
'001	'001	'001	'001	'001	'001	'001	'001	'001	'001	'001	'001	'001
'001	'001	'001	'001	'001	'001	'001	'001	'001	'001	'001	'001	'001
'001	'001	'001	'001	'001	'001	'001	'001	'001	'001	'001	100	100
'001	'001	'001	'001	'001	'001	'001	'001	'001	'001	100	100	100
'001	'001	'001	'001	'001	'001	'001	'001	'001	100	100	100	100
'001	'001	'001	'001	'001	'001	'001	'001	100	100	100	100	100
'001	'001	'001	'001	'001	'001	'001	'001	100	100	100	100	100
'001	'001	'001	'001	'001	'001	'001	'001	'001	100	100	100	100
'001	'001	'001	'001	'001	'001	'001	'001	'001	'001	100	100	100
'001	'001	'001	'001	'001	'001	'001	'001	'001	'001	'001	100	100
'001	'001	'001	'001	'001	'001	'001	'001	'001	'001	'001	'001	100
'001	'001	'001	'001	'001	'001	'001	'001	'001	'001	'001	'001	'001

2. Citra RGB 8 bit

	R (3 bit)	G (3 bit)	B (2 bit)									
\$001267F4	0.5625	3.21875	3.8125	'000	'011	'11	'000 011 11					
\$004F20FF	2.46875	1	3.984375	'010	'001	'11	'010 001 11					
\$00FF005A	7.96875	0	1.40625	'111	'000	'01	'111 000 01					
	2^3	2^3	2^2									
	32	32	64									
Citra 8-bit												
'000 011 11	'000 011 11	'000 011 11	'000 011 11	'000 011 11	'000 011 11	'000 011 11	'000 011 11	'000 011 11	'000 011 11	'000 011 11	'000 011 11	'000 011 11
'000 011 11	'000 011 11	'010 001 11	'000 011 11	'000 011 11	'000 011 11	'000 011 11	'000 011 11	'000 011 11	'000 011 11	'000 011 11	'111 000 01	'000 011 11
'000 011 11	'000 011 11	'010 001 11	'010 001 11	'000 011 11	'000 011 11	'000 011 11	'000 011 11	'000 011 11	'000 011 11	'111 000 01	'111 000 01	'000 011 11
'000 011 11	'000 011 11	'010 001 11	'010 001 11	'010 001 11	'000 011 11	'000 011 11	'000 011 11	'000 011 11	'000 011 11	'111 000 01	'111 000 01	'111 000 01
'000 011 11	'000 011 11	'010 001 11	'010 001 11	'010 001 11	'010 001 11	'000 011 11	'000 011 11	'000 011 11	'000 011 11	'111 000 01	'111 000 01	'111 000 01
'000 011 11	'000 011 11	'010 001 11	'010 001 11	'010 001 11	'010 001 11	'010 001 11	'000 011 11	'000 011 11	'000 011 11	'111 000 01	'111 000 01	'111 000 01
'000 011 11	'000 011 11	'010 001 11	'010 001 11	'010 001 11	'010 001 11	'010 001 11	'000 011 11	'000 011 11	'000 011 11	'111 000 01	'111 000 01	'111 000 01
'000 011 11	'000 011 11	'010 001 11	'010 001 11	'010 001 11	'010 001 11	'010 001 11	'000 011 11	'000 011 11	'000 011 11	'111 000 01	'111 000 01	'111 000 01
'000 011 11	'000 011 11	'010 001 11	'010 001 11	'010 001 11	'010 001 11	'010 001 11	'000 011 11	'000 011 11	'000 011 11	'111 000 01	'111 000 01	'111 000 01
'000 011 11	'000 011 11	'010 001 11	'010 001 11	'010 001 11	'010 001 11	'010 001 11	'000 011 11	'000 011 11	'000 011 11	'111 000 01	'111 000 01	'111 000 01
'000 011 11	'000 011 11	'010 001 11	'010 001 11	'010 001 11	'010 001 11	'010 001 11	'000 011 11	'000 011 11	'000 011 11	'111 000 01	'111 000 01	'111 000 01
'000 011 11	'000 011 11	'010 001 11	'010 001 11	'010 001 11	'010 001 11	'010 001 11	'000 011 11	'000 011 11	'000 011 11	'111 000 01	'111 000 01	'111 000 01

3. Citra RGB 16 Bit

[illegible]

4. Ketik program berikut ini, perhatikan bahwa bitmap di ubah menjadi data matrik bertipe integer agar lebih mudah diproses nantinya

```

1  from PIL import Image
2  citra = Image.open('tulip.png')
3  pixelcitra = citra.load()
4  lebar = citra.size[0]
5  tinggi = citra.size[1]
6  hasil = Image.new(mode="RGB", size=(lebar,tinggi))
7  pixelhasil = hasil.load()
8  bitmapR = []
9  bitmapG = []
10 bitmapB = []
11 bitmapBW = []
12 bitmapGray = []
13 for y in range(0,tinggi):
14     bitmapR.append([])
15     bitmapG.append([])
16     bitmapB.append([])
17     bitmapBW.append([])
18     bitmapGray.append([])
19
20 for x in range(0,lebar):
21     R = pixelcitra[x,y][0]
22     G = pixelcitra[x,y][1]
23     B = pixelcitra[x,y][2]
24     grayscale = round((R+G+B)/3)
25     bitmapR[y].append(R)
26     bitmapG[y].append(G)
27     bitmapB[y].append(B)
28     if grayscale < 128:
29         bitmapBW[y].append(0)
30     else:
31         bitmapBW[y].append(1)
32     bitmapGray[y].append(grayscale)
33 for y in range(0,tinggi):
34     for x in range(0,lebar):
35         pikselhasil = (bitmapR[y][x], bitmapG[y][x], bitmapB[y][x])
36         hasil.show()
37         hasil.save('warna.jpg')

```

Source Code:

```
from PIL import Image

citra = Image.open('flower.png')

pixelcitra = citra.load()
lebar = citra.size[0]
tinggi = citra.size[1]
hasil = Image.new(mode = "RGB", size = (lebar, tinggi))
pixelhasil = hasil.load()

bitmapR = []
bitmapG = []
bitmapB = []
bitmapBW = []
bitmapGray = []

for y in range (0, tinggi):
    bitmapR.append([])
    bitmapG.append([])
    bitmapB.append([])
    bitmapBW.append([])
    bitmapGray.append([])

    for x in range(0, lebar):
        R = pixelcitra[x, y][0]
        G = pixelcitra[x, y][1]
        B = pixelcitra[x, y][2]

        grayscale = round((R + G + B) / 3)
```

```
        bitmapR[y].append(R)

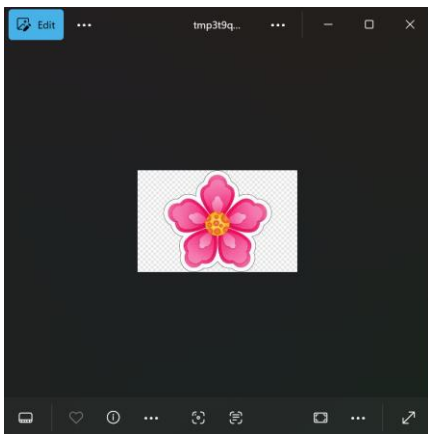
        bitmapG[y].append(G)

        bitmapB[y].append(B)

    if grayscale < 128:
        bitmapBW[y].append(0)
    else:
        bitmapBW[y].append(1)
    bitmapGray[y].append(grayscale)

for y in range(0, tinggi):
    for x in range(0, lebar):
        pixelhasil[x, y] = (bitmapR[y][x], bitmapG[y][x], bitmapB[y][x])
hasil.show()
hasil.save('warna.jpg')
```

Output:



WarnaR:

Source Code:

```
from PIL import Image

citra = Image.open('flower.png')

pixelcitra = citra.load()
lebar = citra.size[0]
tinggi = citra.size[1]
hasil = Image.new(mode = "RGB", size = (lebar, tinggi))
pixelhasil = hasil.load()

bitmapR = []
bitmapG = []
bitmapB = []
bitmapBW = []
bitmapGray = []

for y in range (0, tinggi):
    bitmapR.append([])
    bitmapG.append([])
    bitmapB.append([])
    bitmapBW.append([])
    bitmapGray.append([])

    for x in range(0, lebar):
        R = pixelcitra[x, y][0]
        G = pixelcitra[x, y][1]
        B = pixelcitra[x, y][2]

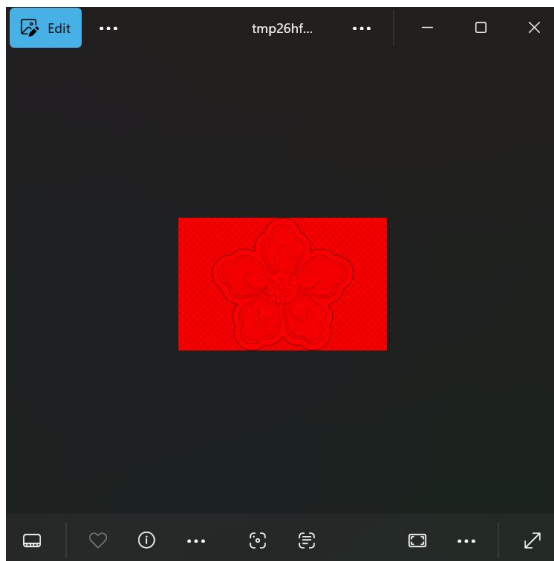
        grayscale = round((R + G + B) / 3)
```

```
        bitmapR[y].append(R)
        bitmapG[y].append(G)
        bitmapB[y].append(B)

    if grayscale < 128:
        bitmapBW[y].append(0)
    else:
        bitmapBW[y].append(1)
    bitmapGray[y].append(grayscale)

for y in range(0, tinggi):
    for x in range(0, lebar):
        pixelhasil[x, y] = (bitmapR[y][x], 0, 0)
hasil.show()
hasil.save('warnaR.jpg')
```

Output:



WarnaG:

Source Code:

```
from PIL import Image
```

```
citra = Image.open('flower.png')

pixelcitra = citra.load()
lebar = citra.size[0]
tinggi = citra.size[1]
hasil = Image.new(mode = "RGB", size = (lebar, tinggi))
pixelhasil = hasil.load()

bitmapR = []
bitmapG = []
bitmapB = []
bitmapBW = []
bitmapGray = []

for y in range (0, tinggi):
    bitmapR.append([])
    bitmapG.append([])
    bitmapB.append([])
    bitmapBW.append([])
    bitmapGray.append([])

    for x in range(0, lebar):
        R = pixelcitra[x, y][0]
        G = pixelcitra[x, y][1]
        B = pixelcitra[x, y][2]

        grayscale = round((R + G + B) / 3)
        bitmapR[y].append(R)
```

```

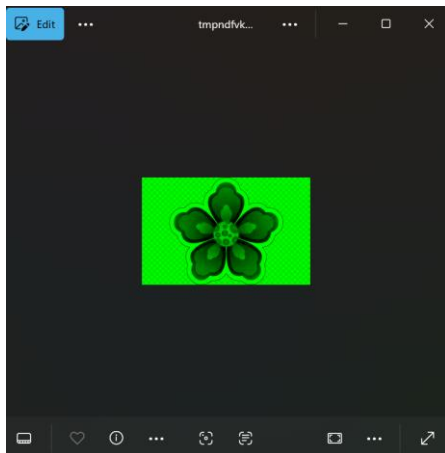
        bitmapG[y].append(G)
        bitmapB[y].append(B)

    if grayscale < 128:
        bitmapBW[y].append(0)
    else:
        bitmapBW[y].append(1)
    bitmapGray[y].append(grayscale)

for y in range(0, tinggi):
    for x in range(0, lebar):
        pixelhasil[x, y] = (0, bitmapG[y][x], 0)
hasil.show()
hasil.save('warnaG.jpg')

```

Output:



WarnaB:

Source Code:

```

from PIL import Image

citra = Image.open('flower.png')

```



```
pixelcitra = citra.load()
lebar = citra.size[0]
tinggi = citra.size[1]
hasil = Image.new(mode = "RGB", size = (lebar, tinggi))
pixelhasil = hasil.load()

bitmapR = []
bitmapG = []
bitmapB = []
bitmapBW = []
bitmapGray = []

for y in range (0, tinggi):
    bitmapR.append([])
    bitmapG.append([])
    bitmapB.append([])
    bitmapBW.append([])
    bitmapGray.append([])

    for x in range(0, lebar):
        R = pixelcitra[x, y][0]
        G = pixelcitra[x, y][1]
        B = pixelcitra[x, y][2]

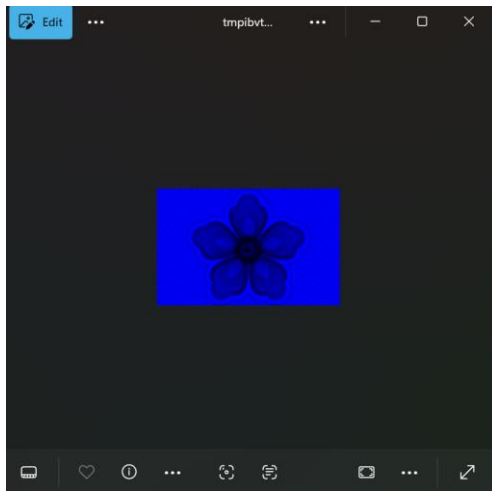
        grayscale = round((R + G + B) / 3)

        bitmapR[y].append(R)
        bitmapG[y].append(G)
        bitmapB[y].append(B)
```

```
        if grayscale < 128:
            bitmapBW[y].append(0)
        else:
            bitmapBW[y].append(1)
        bitmapGray[y].append(grayscale)

for y in range(0, tinggi):
    for x in range(0, lebar):
        pixelhasil[x, y] = (0, 0, bitmapB[y][x])
hasil.show()
hasil.save('warnaB.jpg')
```

Output:



BW:

Source Code:

```
from PIL import Image

citra = Image.open('flower.png')

pixelcitra = citra.load()
```

```
lebar = citra.size[0]
tinggi = citra.size[1]
hasil = Image.new(mode = "RGB", size = (lebar, tinggi))
pixelhasil = hasil.load()

bitmapR = []
bitmapG = []
bitmapB = []
bitmapBW = []
bitmapGray = []

for y in range (0, tinggi):
    bitmapR.append([])
    bitmapG.append([])
    bitmapB.append([])
    bitmapBW.append([])
    bitmapGray.append([])

    for x in range(0, lebar):
        R = pixelcitra[x, y][0]
        G = pixelcitra[x, y][1]
        B = pixelcitra[x, y][2]

        grayscale = round((R + G + B) / 3)
        bitmapR[y].append(R)
        bitmapG[y].append(G)
        bitmapB[y].append(B)

        if grayscale < 128:
```

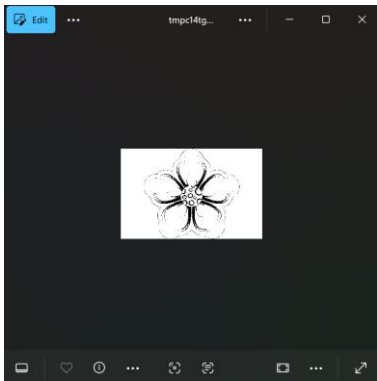
```

        bitmapBW[y].append(0)
    else:
        bitmapBW[y].append(1)
    bitmapGray[y].append(grayScale)

for y in range(0, tinggi):
    for x in range(0, lebar):
        pixelhasil[x, y] = (bitmapBW[y][x]*255, bitmapBW[y][x]*255,
        bitmapBW[y][x]*255)
hasil.show()
hasil.save('biner.jpg')

```

Output:



Grayscale:

Source Code:

```

from PIL import Image

citra = Image.open('flower.png')

pixelcitra = citra.load()
lebar = citra.size[0]
tinggi = citra.size[1]
hasil = Image.new(mode = "RGB", size = (lebar, tinggi))

```

```
pixelhasil = hasil.load()

bitmapR = []
bitmapG = []
bitmapB = []
bitmapBW = []
bitmapGray = []

for y in range (0, tinggi):
    bitmapR.append([])
    bitmapG.append([])
    bitmapB.append([])
    bitmapBW.append([])
    bitmapGray.append([])

    for x in range(0, lebar):
        R = pixelcitra[x, y][0]
        G = pixelcitra[x, y][1]
        B = pixelcitra[x, y][2]

        grayscale = round((R + G + B) / 3)

        bitmapR[y].append(R)
        bitmapG[y].append(G)
        bitmapB[y].append(B)

        if grayscale < 128:
            bitmapBW[y].append(0)
        else:
            bitmapBW[y].append(1)
```

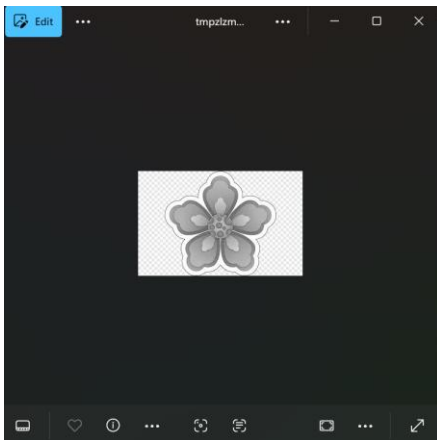
```

        bitmapGray[y].append( grayscale )

for y in range(0, tinggi):
    for x in range(0, lebar):
        pixelhasil[x, y] = ( bitmapGray[y][x], bitmapGray[y][x],
        bitmapGray[y][x] )
hasil.show()
hasil.save('grayscale.jpg')

```

Output:



5. Buatlah program untuk melakukan kuantisasi warna RGB 24 ke RGB 3 bit, RGB 8 bit dan RGB 16 bit

Source Code:

```

from PIL import Image

citra = Image.open('flower.png')
pixel = citra.load()
lebar, tinggi = citra.size

hasil3 = Image.new("RGB", (lebar, tinggi))
p3 = hasil3.load()

for y in range(tinggi):
    for x in range(lebar):
        R, G, B = pixel[x, y]

        Rq = R // 128
        Gq = G // 128
        Bq = B // 128

```

```
        p3[x, y] = (Rq * 255, Gq * 255, Bq * 255)

hasil3.save("rgb_3bit.png")
hasil3.show()

hasil8 = Image.new("RGB", (lebar, tinggi))
p8 = hasil8.load()

for y in range(tinggi):
    for x in range(lebar):
        R, G, B = pixel[x, y]

        Rq = R // 32
        Gq = G // 32
        Bq = B // 64

        p8[x, y] = (Rq * 32, Gq * 32, Bq * 64)

hasil8.save("rgb_8bit.png")
hasil8.show()

hasil16 = Image.new("RGB", (lebar, tinggi))
p16 = hasil16.load()

for y in range(tinggi):
    for x in range(lebar):
        R, G, B = pixel[x, y]

        Rq = R // 8
        Gq = G // 4
        Bq = B // 8

        p16[x, y] = (Rq * 8, Gq * 4, Bq * 8)

hasil16.save("rgb_16bit.png")
hasil16.show()
```

Output:

